TASK 1

Introduction to VLSI Design Flow & Basic Digital Logic Implementation

Part A: VLSI Fundamentals | Part B: Design Flow

Part C: Digital Logic Gates | Part D: Half Adder

1.	**Part A: What is VLSI?**	3
2.	**Part B: VLSI Design Flow (9 Steps)**	4
3.	**Part C: Digital Logic Gates & Truth Tables**	5
4.	**Part D: Half Adder Circuit & Simulation**	8
5.	**Observations & Conclusions**	9

Part A: What is VLSI?

VLSI stands for **Very Large Scale Integration**. It is the process of integrating **millions or billions of transistors** on a single silicon chip. Modern microprocessors, smartphones, AI accelerators, and IoT devices all depend on VLSI technology.

Examples of VLSI Chips

Category	Examples
Microprocessors	Intel Core i9, AMD Ryzen
Mobile SoCs	Qualcomm Snapdragon, Apple M-series
Memory Chips	DDR5 RAM, NAND Flash, ROM
GPUs / AI	NVIDIA H100, Google TPU

Applications of VLSI

- Smartphones and tablets
- Personal computers and laptops
- Automotive electronics (ADAS, ECU)
- Medical devices (pacemakers, MRI)
- AI and machine learning hardware
- IoT and embedded systems

The VLSI design flow is a structured sequence of steps that transforms a design idea into a manufacturable chip. Understanding this flow is essential before writing any HDL code.

Part B: VLSI Design Flow (9 Steps)

Step	Stage	Description
1	**System Specification**	Define overall system requirements and functionality.
2	**Architecture Design**	Plan system structure, modules, and data flow.
3	**RTL Design**	Write Register Transfer Level code to describe logic.
4	**Functional Verification**	Test RTL design to ensure correctness of functionality.
5	**Synthesis**	Convert RTL code into gate-level netlist.
6	**Physical Design**	Map netlist to physical layout of the chip.
7	**Timing & Power Analysis**	Check performance, timing closure, and power efficiency.
8	**Fabrication (Tape-Out)**	Send final design for semiconductor manufacturing.
9	**Testing & Validation**	Verify fabricated chip meets specifications and works reliably.

Part C: Digital Logic Gates & Truth Tables

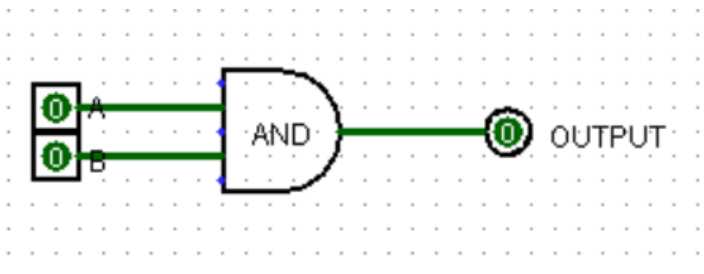
Logic gates are the fundamental building blocks of all digital circuits. Each gate implements a Boolean function.

Below are the six standard gates with their symbols, Boolean expressions, and complete truth tables.

A	B	AND	OR	NOT(A)	NAND	NOR	XOR
0	0	0	0	1	1	1	0
0	1	0	1	1	1	0	1
1	0	0	1	0	1	0	1
1	1	1	1	0	0	0	0

AND Gate: $Y = A \cdot B$

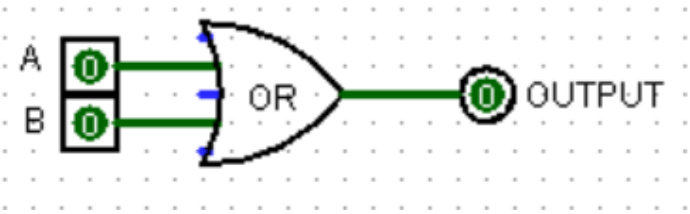
Output is HIGH only when both A and B are HIGH. Used in enable circuits, address decoders.



Inputs		Output
A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate: $Y = A + B$

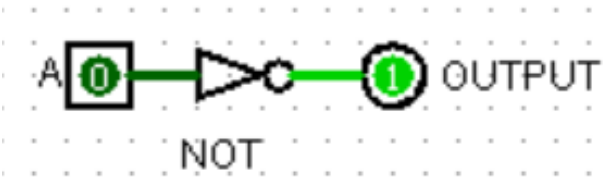
Output is HIGH when at least one input is HIGH. Used in flag registers, interrupt logic.



Inputs		Output
A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate (Inverter): $Y = \bar{A}$

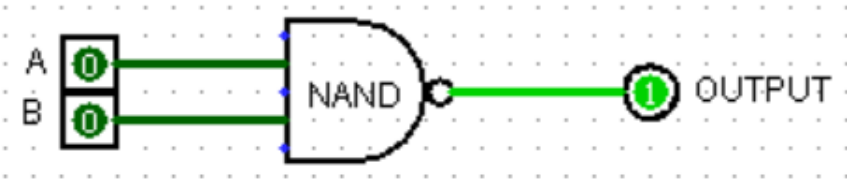
Single-input gate that complements the input. Essential in every sequential and combinational circuit.



Inputs	Output
A	B
0	1
1	0

NAND Gate: $Y = (A \cdot B)'$

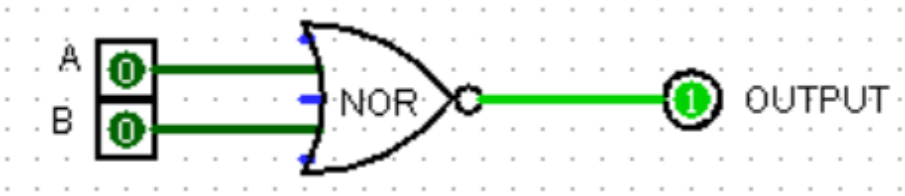
Inverted AND. Functionally complete any Boolean function can be built from NAND gates alone.



Inputs		Output
A	B	$\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate : $Y = (A + B)'$

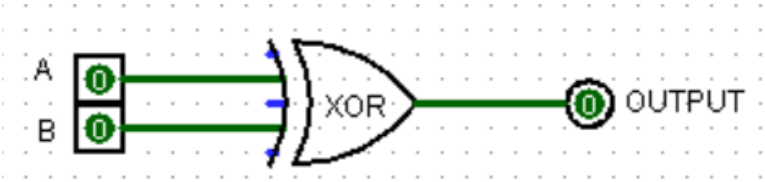
Inverted OR. Also, functionally complete. Used heavily in PLA (Programmable Logic Array) design.



Inputs		Output
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate (Exclusive OR): $Y = A \oplus B$

Output HIGH when inputs DIFFER. Core building block of adders, comparators, and parity checkers.



Inputs		Output
A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Part D: Half Adder — Combinational Circuit

A **Half Adder** is the simplest arithmetic combinational circuit. It adds two single-bit binary numbers (A and B) and produces two outputs: **Sum** and **Carry**.

Boolean Expressions

Output	Expression	Gate Used
SUM (S)	**S = A ⊕ B**	XOR Gate
CARRY (C)	**C = A · B**	AND Gate

Circuit Diagram

Half Adder Circuit Diagram

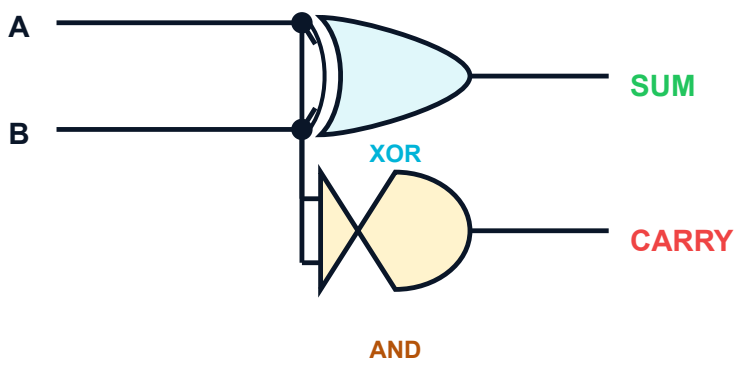
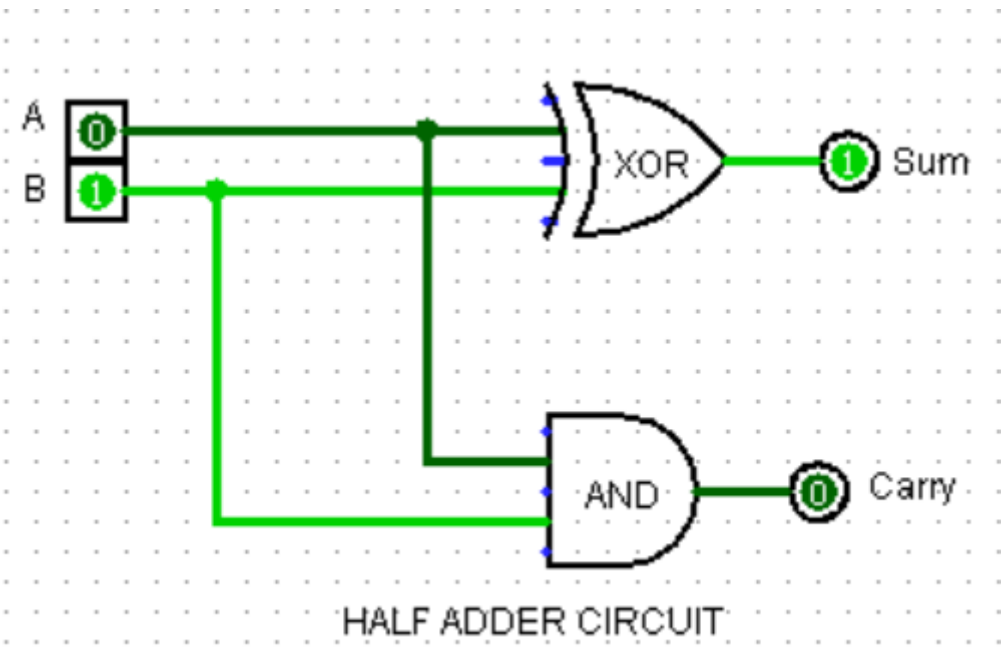


Figure: Half Adder using XOR gate (Sum) and AND gate (Carry)



Truth Table — Half Adder

A (Input)	B (Input)	SUM (A⊕B)	CARRY (A·B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Row-by-Row Explanation

- **A=0, B=0:** No bits are 1. Sum = 0⊕0 = 0. Carry = 0·0 = 0. Result: 0+0 = 0 (no carry).
- **A=0, B=1:** One bit is 1. Sum = 0⊕1 = 1. Carry = 0·1 = 0. Result: 0+1 = 1.
- **A=1, B=0:** One bit is 1. Sum = 1⊕0 = 1. Carry = 1·0 = 0. Result: 1+0 = 1.
- **A=1, B=1:** Both bits are 1. Sum = 1⊕1 = 0. Carry = 1·1 = 1. Result: 1+1 = 10(decimal 2).
-

Observations & Conclusions

Logic Gate Observations

- AND gate: output is 1 in exactly 1 of 4 input combinations (A=B=1).
- OR gate: output is 0 in exactly 1 of 4 input combinations (A=B=0).
- NOT gate: perfectly inverts — output is always opposite of input.
- NAND gate: output is 0 in exactly 1 case, confirming it is the complement of AND.
- NOR gate: output is 1 in exactly 1 case, confirming it is the complement of OR.
- XOR gate: output is 1 only when inputs differ — key property for parity and addition.
- NAND and NOR are universal gates: any logic function can be implemented using only one of them.

Half Adder Observations

- The Half Adder correctly implements binary addition for 1-bit inputs.
- Sum output follows XOR behaviour exactly — verified across all 4 input combinations.
- Carry output follows AND behaviour exactly — carry is generated only when both inputs are 1.
- Maximum binary result is 1+1 = 10 (decimal 2), correctly represented as Sum=0, Carry=1.
- A Full Adder extends this design by adding a third input (carry-in) to handle multi-bit addition.

VLSI Design Flow Observations

- The design flow is sequential — each stage depends on the correctness of the previous one.
- Functional verification (Step 4) is critical; bugs found after synthesis are 10× more expensive to fix.
- RTL (Step 3) is the primary entry point for engineers; skills in Verilog directly map here.
- Physical design (Step 6) introduces real-world constraints: wire resistance, capacitance, and heat.
- Task 1 establishes the conceptual foundation for all subsequent RTL and Verilog-based tasks.

Summary: Task 1 has been completed successfully. All six logic gates have been analysed with verified truth tables and gate diagrams. The Half Adder circuit has been designed using an XOR gate (Sum) and an AND gate (Carry), with all four input combinations confirmed correct. The VLSI design flow from System Specification through Testing & Validation has been documented

— End of Task 1 —